

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



MÔN HỌC: CƠ SỞ LẬP TRÌNH

ĐỒ ÁN MÔN HỌC

Halo - Cyber Access Engine

Giảng viên lý thuyết: Trương Toàn Thịnh
Giảng viên thực hành: Nguyễn Trọng Việt

Sinh viên thực hiện: Mai Thúc Hải Đăng - 24120276

Thành phố Hồ Chí Minh, tháng 6/2026

Mục lục

1	Đánh giá mức hoàn thành đề án	2
2	Thiết kế hệ thống	2
2.1	Cấu trúc dữ liệu	2
2.2	Tiền xử lý đa luồng & Sắp xếp chỉ mục	2
2.3	Khai thác chuỗi sự kiện bằng Sliding Window	4
3	Cách sử dụng	5
4	Các khó khăn gặp phải và Hướng giải quyết	5

1. Đánh giá mức hoàn thành đề án

- Đã hoàn thành 100% các yêu cầu cuối kì.
- Hệ thống có khả năng chịu tải và phân tích siêu tốc trên tập dữ liệu khổng lồ (1 triệu dòng).
- Tự thiết kế cấu trúc dữ liệu với mảng động, không sử dụng các container có sẵn của thư viện.
- Thực hiện đầy đủ, chính xác các chức năng khai thác hệ thống: Tích hợp thành công module phát hiện bất thường (**AnomalyDetector**), có khả năng nhận diện 11 kịch bản vi phạm an ninh khác nhau dựa trên chuỗi sự kiện.

2. Thiết kế hệ thống

2.1. Cấu trúc dữ liệu

Để đảm bảo tốc độ tải dữ liệu triệu dòng nhanh nhất và tiết kiệm tối đa RAM, hệ thống đã load toàn bộ file vào một mảng ký tự duy nhất trên vùng nhớ Heap với mảng `const char*`. Dữ liệu được phân tách bằng cách thay các ký tự phân cách thành `'\0'`, giúp mảng 1 chiều (`fileDataArr`) và mảng 2 chiều (`rows`) chỉ việc lưu trữ **con trỏ** trỏ tới từng từ.

Listing 1: Cấu trúc dữ liệu DataArray

```
1 struct DataArray
2 {
3     char*** rows;
4     char** fileDataArr;
5     char* fileData;
6     string* columnNames;
7     int columnSize;
8     int rowSize;
9 };
```

2.2. Tiền xử lý đa luồng & Sắp xếp chỉ mục

Em đã sử dụng kỹ thuật **Sort by Index** để tránh chi phí hoán đổi mảng lớn. Nhờ đa luồng (`std::thread`), các mảng chỉ mục theo `user_id` và `resource_id` được sắp

xếp song song, đồng thời sử dụng `std::stable_sort` để bảo toàn tính thứ tự của `timestamp`. Tương tự đối với module `AnomalyDetector`, em đã định nghĩa thêm `UserIdx` và `DeviceIdx` để phục vụ phân tích.

Listing 2: Khởi tạo cấu trúc và sắp xếp đa luồng trong `AnomalyDetector`

```
1 struct UserIdx
2 {
3     const char* user_id;
4     unsigned long long timestamp;
5     int idx;
6 };
7
8 struct DeviceIdx
9 {
10    const char* device_id;
11    unsigned long long timestamp;
12    int idx;
13 };
14
15 thread prepareThread([&]()
16 {
17     for (int i = 0; i < datas.rowSize; ++i)
18     {
19         uArr[i].user_id = datas.rows[i][userIdxCol];
20         uArr[i].timestamp = fast_stoull(datas.rows[i][
21             timeIdxCol]);
22         uArr[i].idx = i;
23
24         dArr[i].device_id = datas.rows[i][devIdxCol];
25         dArr[i].timestamp = uArr[i].timestamp;
26         dArr[i].idx = i;
27     }
28
29     thread t1([&]()
30     {
31         stable_sort(uArr, uArr + datas.rowSize, [](const
32             UserIdx& a, const UserIdx& b)
33         {
34             return strcmp(a.user_id, b.user_id) < 0;
35         });
36     });
37 }
```

```

34     });
35
36     thread t2([&]())
37     {
38         stable_sort(dArr, dArr + datas.rowSize, [](const
39             DeviceIdx& a, const DeviceIdx& b)
40         {
41             return strcmp(a.device_id, b.device_id) < 0;
42         });
43
44     t1.join();
45     t2.join();
46 });

```

2.3. Khai thác chuỗi sự kiện bằng Sliding Window

Nhờ quá trình gom nhóm đối tượng ở bước tiền xử lý, việc truy vết bất thường trở nên dễ dàng. Em đã duyệt mảng chỉ mục bằng Cửa sổ trượt với độ phức tạp $O(N)$ để bắt các mẫu hành vi như: Đăng nhập thất bại liên tục, Một thiết bị quét quá nhiều Resource trong 1 phút, Người dùng thay đổi vị trí địa lý phi lý, v.v.

Listing 3: Thuật toán quét cửa sổ bắt chuỗi sự kiện Login thất bại liên tục

```

1  int failedCount = 0;
2  unsigned long long firstFailTime = 0, lastFailTime = 0;
3  for (int current = start; current < end; ++current) {
4      if (strcmp(datas.rows[uArr[current].idx][evtIdxCol], "
5          FAILED_LOGIN") == 0) {
6          if (failedCount == 0) firstFailTime = uArr[current].
7              timestamp;
8              lastFailTime = uArr[current].timestamp;
9              failedCount++;
10     } else if (strcmp(datas.rows[uArr[current].idx][evtIdxCol],
11         "LOGIN") == 0) {
12         if (failedCount >= 5) {
13             results.push_back("User_" + string(uArr[start].
14                 user_id) + "_fail_login_lien_tuc_" + to_string(
15                 failedCount) + ".lan");
16         }
17         failedCount = 0;

```

13

}

14

}

3. Cách sử dụng

Trước khi chạy file thực thi, copy file dữ liệu và đổi tên thành "**data.csv**" đặt trong thư mục **release**. Sau khi khởi chạy, hệ thống sẽ đọc nguyên khối, tiền xử lý và hiển thị Menu gồm các tính năng:

1. **Hành trình Device → App → Resource của 1 User**: Nhập User ID, thời gian bắt đầu và kết thúc để truy vết.
2. **Hành trình User → Device → App của 1 Resource**: Nhập Resource ID, thời gian bắt đầu và kết thúc để truy vết.
3. **Top 10 Resource được truy xuất nhiều nhất**: Nhập thời gian bắt đầu và kết thúc để nhận bảng xếp hạng Top 10 tài nguyên có lượng truy cập cao nhất.
4. **Phát hiện hành vi bất thường**: Mở ra Sub-menu chứa 11 chức năng truy vết các lỗi hỏng bảo mật (fail login, spam thiết bị, truy cập ngoài giờ, dịch chuyển phi lý,...). Kết quả được xuất ra theo cơ chế phân trang (xem mỗi 1000 dòng) để người dùng dễ theo dõi.
5. **Thoát và giải phóng bộ nhớ**: Dọn dẹp toàn bộ Heap và đóng chương trình một cách an toàn.

4. Các khó khăn gặp phải và Hướng giải quyết

1. Tốc độ chậm khi hoán vị dòng dữ liệu cồng kềnh: Việc sắp xếp trực tiếp lên mảng 2 chiều là bất khả thi với dữ liệu 1 triệu dòng do quá trình hoán vị nặng nề nên em đã áp dụng thuật toán **Sort by Index**. Dùng các struct nhẹ như **NumIdx**, **StrIdx** để thao tác hoán vị, chỉ đổi chiều mảng gốc khi cần trích xuất.
2. Bé tắc khi dò tìm bất thường trên chuỗi ngẫu nhiên: Các sự kiện bị phân mảnh rải rác trên file 1 triệu dòng khiến việc thống kê từng User rất tốn kém ($O(N^2)$) nên em đã tận dụng đa luồng để sắp xếp các mảng chỉ mục hoàn toàn theo ID (**UserIdx**, **DeviceIdx**) giúp gom các sự kiện của cùng 1 đối tượng về sát nhau. Sau đó chỉ cần 1 vòng lặp **Sliding Window** $O(N)$ lướt qua là có thể bắt được bất kỳ chuỗi hành động đáng ngờ nào.